

Katedra informatiky
Přírodovědecká fakulta
Univerzita Palackého v Olomouci

BAKALÁŘSKÁ PRÁCE

Webová aplikace s texty Bible



2026

Vedoucí práce:
Mgr. Tomáš Urbanec, Ph.D.

Patrik Prinz

Studijní program: Informatika,
Specializace: Programování a vývoj
software

Bibliografické údaje

Autor:	Patrik Prinz
Název práce:	Webová aplikace s texty Bible
Typ práce:	bakalářská práce
Pracoviště:	Katedra informatiky, Přírodovědecká fakulta, Univerzita Palackého v Olomouci
Rok obhajoby:	2026
Studijní program:	Informatika, Specializace: Programování a vývoj software
Vedoucí práce:	Mgr. Tomáš Urbanec, Ph.D.
Počet stran:	??
Přílohy:	elektronická data v úložišti katedry informatiky
Jazyk práce:	český

Bibliographic info

Author:	Patrik Prinz
Title:	Web application with Bible texts
Thesis type:	bachelor thesis
Department:	Department of Computer Science, Faculty of Science, Palacký University Olomouc
Year of defense:	2026
Study program:	Computer Science, Specialization: Programming and Software Development
Supervisor:	Mgr. Tomáš Urbanec, Ph.D.
Page count:	??
Supplements:	electronic data in the storage of department of computer science
Thesis language:	Czech

Anotace

Text se zaměřuje na popis vývoje a fungování webové aplikace, která obsahuje texty Bible a žaltáře určené pro Řeckokatolickou farnost v Olomouci. Zaměřuje se na analýzu požadavků, realizaci projektu a blíže popisuje použité koncepty z oblasti DevOps.

Synopsis

This text focuses on development and description of functions of web application with Bible and psalter texts. This application is intended for use by Easter catholic parish in Olomouc. It more closely describes a process of requirement analysis, realization of the project and usage of DevOps concepts during the development process.

Klíčová slova: webová aplikace; Bible; PWA; TypeScript; DevOps; CI/CD

Keywords: web application; Bible; PWA; TypeScript; DevOps; CI/CD

Tímto bych rád poděkoval vedoucímu mé práce, Mgr. Tomáši Urbanci PhD., za poskytnutý čas a cenné rady při jejím zpracování. Dále bych také chtěl poděkovat faráři Řeckokatolické farnosti v Olomouci, otci Petru Prinzovi za podporu při realizaci projektu a inspiraci k ní.

Odevzdáním tohoto textu jeho autor místopřísežně prohlašuje, že celou práci včetně příloh vypracoval samostatně a za použití pouze zdrojů citovaných v textu práce a uvedených v seznamu literatury.

Obsah

Seznam obrázků

1 Úvod

Tato kapitola se věnuje objasnění základních pojmů z domény projektu a dále vytyčuje cíl a motivaci práce.

1.1 Slovníček pojmů

Vzhledem k oblasti, na kterou se text zaměřuje, budu používat pojmy, které jsou spojeny s církevním prostředím, různými modlitbami a liturgickými texty. Na úvod tedy uvádím sekci, která čtenáře s těmito pojmy stručně seznámí.

1.1.1 Bible

Bible je klíčovou posvátnou knihou křesťanské církve. Jejím studiu, výkladům, vytváření překladů a pojednání o ní je věnováno velké úsilí. Četba, studium a rozjímání textů Bible je důležitou součástí života každého křesťana. Proto existuje v překladech do mnoha jazyků a většina jazyků má překladů více.

Bible je souborem knih různých autorů z historie křesťanství a judaismu. Tyto knihy jsou rozděleny do dvou částí - do **Starého a Nového zákona**. Každá kniha je tvořena kapitolami a kapitoly jsou tvořeny verši.

Významnými knihami jsou 4 evangelia, která popisují život Ježíše Krista a jsou často používány i při liturgických obřadech.

Další významnou knihou je kniha žalmů (takzvaný **žaltář**). Mezi východními křesťanskými církvemi se žaltář rozděluje do 20 částí, kterým se říká **kaftizmy**. Každá kaftizma obsahuje skupinu žalmů a další přidružené texty modliteb. Jednotlivé žalmy a kaftizmy jsou často používány jako samostatné modlitby.

1.2 Cíl práce

Cílem práce je vytvořit aplikaci, která bude uživatelům usnadňovat přístup k modlitebním textům a textům Bible pro každodenní použití. Řeckokatolická farnost v Olomouci, pro kterou bude tato aplikace určena, používá v současné době systém, který neposkytuje dostatečný komfort pro správu obsahu z pohledu administrátora. Systém v současné podobě není dostatečně flexibilní, aby mohl být uspokojivě rozšířen.

1.3 Cílová skupina

Primární cílovou skupinou této aplikace jsou členové Řeckokatolické farnosti v Olomouci. Patří sem běžní farníci, lidé, kteří jsou různým způsobem zapojeni do chodu farnosti, a duchovní. Farnost je v současnosti vícenárodnostní a v posledních letech se značně rozrůstá. Nejpočetněji jsou zde zastoupeni farníci z Ukrajiny.

Farnost má vlastní společenství, které se modlí modlitby žalmů a funguje od roku 2024. Každý člen společenství se během týdne modlí jednu část žaltáře zvanou kaftizma.

1.4 Doména

Hlavní doménou projektu jsou modlitby a církevní texty používané řeckokatolickou církví.

Pro Apoštolský exarchát České republiky, který je nadřazenou organizací olomoucké farnosti, existuje kalendář, který obsahuje i biblické texty pro každý den v roce. Každému dni v roce jsou přiřazeny vybrané texty ze čtyř knih evangelií. Evangelia tvoří pro katolickou církev nejvýznamnější část Bible. Dále je každému dni také přiřazen úryvek z jedné z ostatních knih Nového zákona. Při slavnostních bohoslužbách se biblických textů předčítá větší množství.

Biblické texty v církevních obřadech jsou často vázány ke svátkům, které jsou buď pevně svázány s datem, nebo jsou pohyblivé a připadají každý rok na jiné datum. Mnoho textů je také svázáno s konkrétním obřadem nebo událostí (např. křest, svěcení domu, ...).

1.5 Přínos práce

V rámci této práce se zaměřuji především na implementaci čtečky Bible a textů žaltáře. Aplikace poskytuje Bibli samotnou v různých překladech pro čtení a studium, a také poskytuje vizualizaci úryvků textu podle dostupného kalendáře. Dále je k dispozici i zobrazení textů žaltáře pro osobní modlitbu, nebo modlitbu ve společenství.

Další částí je možnost vytváření skupin, které budou využity společenstvími ve farnosti a možnost sdílení příspěvků se členy těchto skupin. Tato aplikace má za cíl nahradit stávající systém[grekokat-casoslov], který je používán pro poskytování modlitebních textů.

Mým dalším cílem je navrhnout aplikaci s ohledem na rozšiřitelnost, což by umožnilo v budoucnu přidávat další texty, které budou moci fungovat jako samostatné moduly aplikace. Textů je velké množství a celá řada z nich má mnoho složitých pravidel, podle kterých se skládají konkrétní texty pro specifický den, období nebo svátek.

V této práci zaměřuji i na průzkum a využití konceptů z oblasti DevOps a CI/CD, které usnadní vývoj samotný. V dlouhodobém měřítku tyto koncepty urychlí integraci i samotné nasazení aplikace. Zaměřil jsem se na proces ověření kvality a správnosti, nasazení software a sledování jeho stavu. I za cenu větší náročnosti přípravy a počáteční konfigurace je pro mne výhodné získat značně větší spolehlivost a rychlost nasazení nových verzí a také do budoucna usnadnit údržbu potřebné infrastruktury. Díky této filosofii se mohu i v jednočlenném týmu, po nasazení infrastruktury, soustředit více na roli programátora bez nutnosti stále ručně udržovat vše potřebné pro chod projektu. Také začlenění nových

příspěvatelů do projektu může být těmito koncepty značně usnadněno. Díky jednoduchému nastavení všech potřebných částí vývojového prostředí je jednodušší začít pracovat se zdrojovými kódy a vidět i výsledky změn velmi rychle. Díky automatizované kontrole kvality také dostává vývojář zpětnou vazbu o kvalitě provedených změn a jejich kompatibilitě se zbytkem projektu.

2 Podobné projekty

V rámci této práce jsem hledal a procházel i další software zaměřený na práci s texty Bible i dalšími liturgickými texty. Aplikací s tímto zaměřením existuje celá řada. Většina je však určena k jiným účelům a především pro osobní použití, ne pro použití ve farnosti samotné, případně v jiném společenství. Podobné projekty bývají zaměřeny na samotné čtení a studium biblických textů. Většina z projektů má také uzavřený zdrojový kód a proto je nelze upravit a přizpůsobit pro specifické potřeby, nebo je již zastaralá a dlouhodobě neudržovaná. Dále existuje software zaměřený na texty modliteb samotné. Ty však většinou obsahují pouze samotné modlitby, typicky zobrazované podle dne. Projekt, který vyvíjím v rámci této práce, sdružuje a implementuje více specifických požadavků, které ve většině případů nespadají do stejné domény. Pro případ chodu farnosti, nebo jiného společenství je však výhodné je sdružit do jednoho projektu. V seznamu níže uvádím některé ze souvisejících projektů, se kterými jsem se dosud setkal.

2.1 Časoslov.sk

Časoslov.sk[casoslov.sk] je projekt vyvíjený Radou pre mládež košickej eparchie na Slovensku. Poskytuje texty pro jednotlivé dny v roce. Má také knihovnu s různými připravenými texty k modlitbě. Nabízí webovou i mobilní aplikaci s obdobnými funkcionalitami. Časoslov.sk se zaměřuje především na samotné texty pro jednotlivé dny a obsahuje také knihovnu liturgických textů a různých modliteb, jejichž texty se nemění podle dne. Pro věřící v České republice je také klíčová možnost spravovat zároveň více překladů textů, kterou tato aplikace neposkytuje. Neposkytuje také možnost samostatného čtení a studia biblických textů, správu více překladů, ani funkcionalitu skupin.

2.2 BibleGateway

BibleGateway[[biblegateway](http://biblegateway.com)] je webová aplikace sloužící ke studiu biblických textů. Její doména se omezuje pouze na samotné procházení textů Bible a neimplementuje funkcionality jako modlitební skupiny, nebo žaltář a další modlitby. Aplikace na mnoha místech nabízí uživateli placenou verzi, což působí rušivě a není v souladu s mými požadavky.

2.3 Beblia.com

Beblia.com[**beblia**] je další webová aplikace, která nabízí čtečku Biblických textů. Obsahuje také velké množství překladů a v rámci projektu je k dispozici i databáze různých překladů v XML formátu. Texty však nemají přiloženou žádnou licenci, takže není možné je volně používat bez souhlasu autora.

3 Analýza požadavků

Software je určen především pro Řeckokatolickou farnost v Olomouci, její farníky a osoby, které se podílejí na chodu života farnosti. Hlavním cílem je vytvořit platformu, která bude sdružovat Biblické texty, modlitby a informace pro členy společenství, což usnadní práci všech, kteří se na chodu jednotlivých společenství podílejí. Možnost vytvoření kalendáře může být v budoucnu přínosem i pro další řeckokatolíky v České republice mimo tuto farnost. Sekce modliteb by měla být do budoucna rozšiřitelná, protože modliteb, které mohou být zahrnuty, je velké množství a některé z nich se mění podle svátku a období, což může značně komplikovat jejich strukturu. Vzhledem ke složení farnosti je také důležitá možnost použití jazykových mutací.

3.1 Funkční požadavky

Sekce Bible

- Aplikace bude umožňovat zobrazení textů Bible.
- Aplikace bude umožňovat administrátorům importovat překlady ve formátu Zefania XML.
- Aplikace bude umožňovat přihlášeným uživatelům vytvářet si záložky.
- Aplikace bude umožňovat přihlášeným uživatelům označovat verše a přidávat k nim poznámky.

Uživatelské účty

- Aplikace bude implementovat uživatelské účty pro identifikaci uživatelů.
- Aplikace bude implementovat správu rolí, pomocí kterých bude možné přiřazovat uživatelům oprávnění.
- Uživatel si může vytvořit uživatelský účet.
- Uživatel se může pomocí emailu a hesla přihlásit do aplikace.

Skupiny

- Oprávněný uživatel může vytvářet nové skupiny, přidávat do nich členy a přiřazovat jim role v rámci skupiny.

- Oprávněný uživatel může přidávat příspěvky do skupiny.
- Uživatelé mohou zobrazovat příspěvky ve skupině.

Žaltář

- Aplikace bude zobrazovat texty žaltáře po jednotlivých žalmech nebo po kaftizmách.

Úryvky

- Aplikace bude umožňovat přihlášeným uživatelům spravovat svou knihovnu úryvků.
- Oprávněný uživatel může spravovat úryvky, které jsou přiřazeny k jednotlivým dnům.
- Všichni uživatelé mohou zobrazovat texty vložené do kalendáře.

Ostatní

- Aplikace bude umožňovat přepínání jazyka uživatelského rozhraní.

3.2 Nefunkční požadavky

- Aplikace by měla být snadno rozšiřitelná a udržovatelná.
- Aplikace by měla být schopna spravovat své závislosti a co nejméně záviset na prostředí a konfiguraci dané aktuálním stavem systému, na kterém bude spuštěna.
- Aplikace by měla umožňovat snadnou úvodní konfiguraci a spuštění.

4 Návrh řešení

V této sekci se věnuji technologiím a obecným konceptům, které jsem se rozhodl pro realizaci tohoto projektu použít.

4.1 Použité technologie

Vzhledem k charakteru projektu a skutečnosti, že jsem se rozhodl pro využití DevOps postupů, jsem použil celou řadu nástrojů a knihoven, které vývoj značně usnadnily.

4.1.1 Node.js

Node.js[**nodejs**] je běhové prostředí pro jazyk JavaScript. Node lze využít pro spuštění JavaScriptu na jiných platformách než v prohlížeči, pro který byl jazyk původně určený.

Tuto technologii jsem se rozhodl použít pro backend projektu, protože její volba mi umožňuje použít stejný jazyk pro vývoj frontendu i backendu. To mi usnadnilo nastavení nástrojů pro správu kvality kódu a typovou kontrolu. Všechny balíčky tak mohou sdílet velkou část nastavení. I když je rozdíl mezi funkcí JavaScriptu v prohlížeči a v Node.js, základní koncepty a mechanismy jazyka zůstávají stejné.

4.1.2 Pnpm

Pnpm[**pnpm**] je správce balíčků pro Node.js. Pnpm má podobné rozhraní jako npm. Oproti tomuto správci však poskytuje mnoho optimalizací rychlosti instalace a využití místa na disku. Dále také poskytuje možnost správy závislostí vnořených balíčků (takzvané workspaces), což značně usnadňuje práci s mono-repozitářem¹, který v projektu používám. Závislosti je tak možné sdílet mezi balíčky, nebo nainstalovat jen pro konkrétní balíček. Pnpm zajišťuje, že závislosti použité vícekrát se nainstalují pouze jednou. Pnpm používá pro instalaci balíčků sdílený globální adresář a na tyto závislosti odkazuje. Důsledkem této optimalizace je značně efektivnější využití místa na disku.

4.1.3 TypeScript

TypeScript[**typescript**] je staticky typovaný jazyk, který se překládá do JavaScriptu. Jako hlavní jazyk, ve kterém jsem projekt vytvářel, jsem jej zvolil pro jeho univerzalitu. Po přeložení je vhodný pro použití na backendu v prostředí výše zmíněného Node.js i na frontendu aplikace pro běh v prohlížeči. Typescript také na rozdíl od JavaScriptu poskytuje statické typování a s ním i typovou kontrolu, což v mnoha případech značně zjednodušuje vývoj, poskytuje vhodnější automatické doplňování kódu a odhaluje řadu chyb, které při vytváření kódu vznikají, ale bez typové kontroly by se projeví až při běhu nebo testování. Typescript přidává nutnost kompilace projektu, což komplikuje počáteční nastavení projektu. Výsledná typová kontrola je však velmi výhodná a v případě projektu, který je rozsáhlejší, nebo se očekává delší doba životnosti, se počáteční komplikace mohou velmi vyplatit.

4.1.4 Express

Express.js[**express**] je framework pro vývoj webových aplikací i API v JavaScriptu a TypeScriptu. Poskytuje vlastní implementaci routeru, nástroje pro práci s

¹Monorepozitář je způsob strukturování při vývoji software, kdy je více projektů spojeno do jednoho repozitáře a správa verzí se pro ně realizuje společně.

HTTP požadavky a jejich zpracování. Díky minimalistickému návrhu jej lze snadno integrovat s mnoha rozšířeními i dalšími balíčky. Express, na rozdíl od většiny frameworků podobného zaměření, nevynucuje téměř žádné vlastní konvence struktury projektu.

Pro vývoj tohoto projektu jsem Express zvolil především díky jeho jednoduchosti použití a volnosti při návrhu, kterou nabízí. Express je také velmi rozšířený, stabilní a poskytuje snadno použitelné rozhraní.

4.1.5 Vue.js

Vue.js[**vuejs**] je framework pro vytváření webových uživatelských rozhraní. Vue.js Umožňuje uživateli vyvíjet aplikace s architekturou MVVM[**vue-mvvm**]. Vue.js samo o sobě implementuje ViewModel část jako obousměrné provázání dat aplikace s uživatelským rozhraním.

Vue.js také usnadňuje rozdělení uživatelského rozhraní do komponent, které jsou základními stavebními bloky aplikace. Komponenta sdružuje HTML kód, JavaScriptový kód a CSS styl do jednoho souboru.

Vue.js má velmi přehlednou a obsáhlou dokumentaci. Systém komponent a mnoho hotových komponent a knihoven, které jsou s Vue.js kompatibilní, poskytují velmi pohodlný vývoj aplikací.

Před vývojem tohoto projektu jsem neměl mnoho zkušeností s vývojem frontendu za použití obdobných frameworků. Hlavní volbou pro tuto variantu tak byl jednoduchý proces učení a dostatek dostupné dokumentace spolu s rozsáhlým ekosystémem kompatibilních knihoven a rozšíření.

4.1.6 Pinia

Pinia[**pinia**] je knihovna rozšiřující Vue.js. Poskytuje takzvané story, které umožňují sdílet globální stav mezi jednotlivými komponentami, které jsou od sebe odděleny. Pinia store poskytuje funkce, které mohou být použity jako gettery a settery pro vlastnosti, které jsou ve storu uloženy.

Pinia je v současnosti doporučeným řešením pro správu globálního stavu v aplikacích založených na Vue.js a má oficiální podporu.

4.1.7 Elasticsearch

Elasticsearch[**elasticsearch**] je vyhledávací engine, který jsem se rozhodl použít k indexaci a vyhledávání v textech Bible a k jejich uložení. Aplikace může obsahovat libovolné množství různých překladů a Elasticsearch usnadňuje vyhledávání v nich.

4.1.8 PostgreSQL

PostgreSQL[**postgresql**] je v současnosti velmi oblíbenou volbou při výběru relační databáze. PostgreSQL je velmi výkonná a je bez problémů kompatibilní s ostatními technologiemi, které používám. Relační model dat je vhodný pro

uložení dat o uživatelských účtech a jiných dat, která mají pevnou strukturu. PostgreSQL také umožňuje ukládání JSON dokumentů, což jsem také v projektu úspěšně využil.

4.1.9 Kysely

Kysely[**kysely**] je knihovna, která zajišťuje vytváření dotazů a spravuje připojení k databázi. Je použitelná s různými SQL databázovými systémy. Jednoduše se dá využít i pro připojení k databázi PostgreSQL.

Knihovnu Kysely jsem zvolil pro její jednoduchou integraci do projektu. Poskytuje také dobrou dokumentaci a jednoduché objektové rozhraní pro vytváření dotazů. Pomocí typové kontroly TypeScriptu Kysely poskytuje i kontrolu názvů tabulek, sloupců a jejich typu, což značně usnadňuje vývoj.

4.1.10 Docker

Docker[**docker**] je kontejnerizační nástroj, který umožňuje programátorovi vytvářet takzvané kontejnery, což jsou balíčky, které mohou sloužit k distribuci aplikací.

Díky kontejnerizaci odpadá potřeba řešit manuálně všechny závislosti projektu a hlídat jejich verze a nastavení. Všechna potřebná konfigurace je obsažena v docker obrazu, jehož spuštěním vzniká samotný kontejner.

Díky tomu jsem mohl vývoj projektu provádět z více zařízení bez větších komplikací s nastavením vývojového prostředí. Podstatně snadnější je i nasazení aplikace do provozu díky integraci Dockeru do CI/CD pipeline, které se budu dále v textu věnovat.

Souběh kontejnerů zajišťuje Docker Compose. Jde o nástroj, který konfiguruje a spouští kontejnery a další prvky infrastruktury Dockeru na základě jednoduchých konfiguračních souborů.

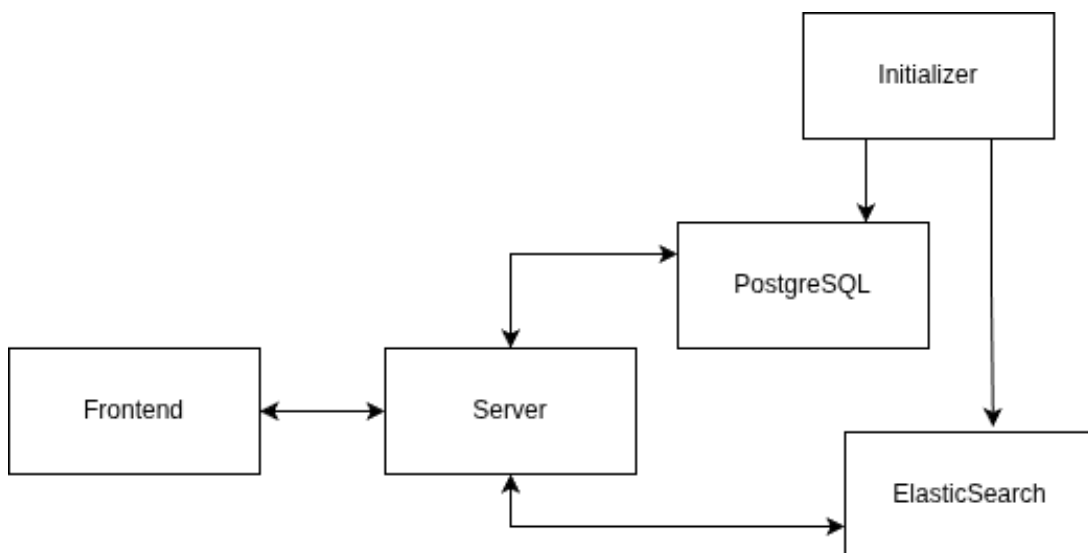
4.1.11 GitHub a GitHub Actions

GitHub jsem zvolil především pro jeho rozšířenost a řadu možností, které poskytuje. Mezi ně patří především služba GitHub Actions, kterou jsem použil pro vytvoření CI/CD pipeline.

GitHub Actions[**github__actions**] je flexibilní nástroj, který umožňuje spouštět akce, jako reakci na události v repozitáři pomocí jednoduchých deklarativních konfigurací. Vzhledem k rozšířenosti tohoto nástroje je k dispozici rozsáhlý ekosystém komponent, které je možné do akcí zapracovat. GitHub Actions je díky tomu snadné integrovat s ostatními nástroji, které jsem pro usnadnění vývoje použil.

4.2 Architektura projektu

Projekt je složen z 5 hlavních komponent, které zajišťují uchování dat a jejich manipulaci. Blokový diagram komponent projektu lze vidět na obrázku ???. Jednotlivým komponentám se budu blíže věnovat v dalších sekcích.



Obrázek 1: Blokové schéma projektu

4.3 Návrh databáze

Aplikace je určena především k prezentaci uložených dat a jejich správě. Proto pro mne byla volba vhodného nástroje důležitá. Pro potřeby projektu jsem se rozhodl využít dva nástroje pro různé účely.

4.3.1 PostgreSQL

PostgreSQL databázi jsem využil jako primární úložiště dat, která bude aplikace uchovávat. Jsou zde uložena data o uživatelích, skupinách, oprávněních, úryvcích biblických textů a další. Důležitějším částem databáze se budu v této části věnovat podrobněji.

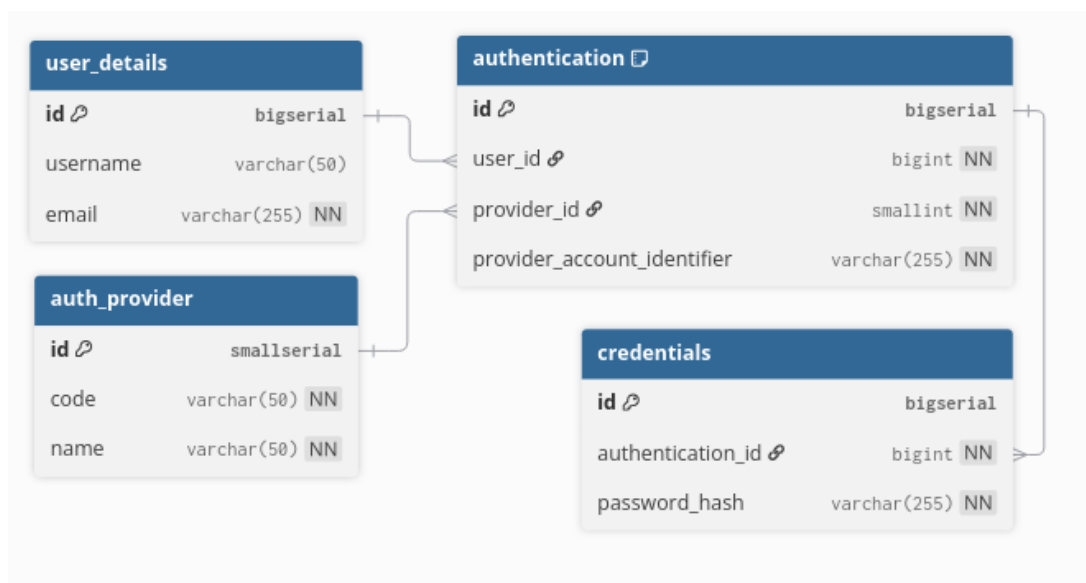
4.3.1.1 Uživatelské účty Databáze obsahuje strukturu pro ukládání dat o uživatelských účtech a možnostech přihlášení uživatele (vizte obrázek ??). Klíčovou tabulkou je tabulka `user_details`, která obsahuje detaily o uživateli. Je zde uložen email, který slouží jako identifikátor uživatele.

Struktura této části databáze je přizpůsobena tomu, aby bylo možné implementovat různé typy přihlašování.

Další tabulkou je tabulka poskytovatelů autentizace. Zde je uložen název a zkratka poskytovatele.

Tabulka authentication obsahuje cizí klíče odkazující na záznam v tabulce poskytovatelů autentizace a dále na záznam z tabulky s detaily uživatele, jehož totožnost se ověřuje a identifikační kód poskytnutý autentizační službou.

Pro přihlášení pomocí hesla existuje tabulka credentials. Zde se ukládá uživatelské heslo. Hodnota provider_account_identifier je poskytována externím poskytovatelem. Pro lokální přihlášení pomocí emailu a hesla je hodnota provider_account_identifier nastavena na local<user_id>.



Obrázek 2: Schéma tabulek pro správu uživatelského účtu a přihlášení

4.3.1.2 Uživatelská metadata Bible Další část databáze tvoří tabulky pro ukládání metadat Bible (ukázka schématu je na obrázku ??). Tato metadata zahrnují záložky, poznámky k veršům Bible a zvýraznění veršů. Všechna tato metadata jsou vyhrazena pro přístup pouze pro jejich autora. Metadata jsou uzpůsobena tak, aby korespondovala s mentálním modelem záložek a poznámek v papírové knize. Tomu odpovídá i reprezentace v databázi a následná práce s nimi.

Záložka představuje pojmenované místo v Bibli. Záložky jsou uzpůsobeny k přesouvání mezi verši a předpokládá se, že název se nemění. S jedním veršem může být v jednu chvíli spojeno i několik záložek.

Ostatní metadata jsou vázána staticky ke konkrétním veršům a není možné je přesunout k jinému verši. V tom případě dochází k jejich odstranění a opětovnému vytvoření. Ke každému místu a autorovi smí v danou chvíli patřit jen jeden záznam v databázi. Proto, že poznámka i zvýraznění jsou vázána ke konkrétnímu verši, jsou uloženy ve stejném záznamu, pokud mají stejné umístění.

Data jsou rozdělena do dvou tabulek. První z nich obsahuje data o záložkách. Záznam v databázi obsahuje informaci o poloze záložky v textu, id autora a

název. Druhá tabulka obsahuje data o poznámkách a zvýrazněných verších. V této tabulce je stejná část popisující autora a umístění. Dále tabulka obsahuje možnost vložit kód barvy zvýraznění, nebo hodnotu NULL a dále také volitelný text poznámky.

user_bookmarks		bible_user_metadata	
id	bigserial	id	bigserial
bible_translation	varchar(10) NN	bible_translation	varchar(10) NN
bible_book	smallint NN	bible_book	smallint NN
bible_chapter	smallint NN	bible_chapter	smallint NN
bible_verse	smallint NN	bible_verse	smallint NN
author_id	bigint	author_id	bigint
bookmark_name	varchar(50) NN	highlight_color	varchar(10)
		note_text	text

Obrázek 3: Schéma tabulek pro správu uživatelských poznámek a záložek k Bibli

4.3.2 Elasticsearch

Pro uložení textů, ve kterých se bude vyhledávat jsem použil Elasticsearch. Elasticsearch je určený primárně k indexaci dat a vyhledávání v nich. Pro účely práce jsem do něj uložil data tvořená větším množstvím textu, ve kterých budu provádět vyhledávání.

4.3.2.1 Texty Bible Uložení textů Bible se budu věnovat podrobněji. Strukturu indexu ukazuje ??.

V databázi jsou verše reprezentovány jako jednotlivé verše. Hierarchickou strukturu kvůli lepší indexaci ukládám po jednotlivých verších s jejich přesným umístěním. Záznam tak obsahuje kód překladu, ke kterému patří, číslo knihy, kapitoly a verše. Dále obsahuje samotný text, který se indexuje a vlastnost `is_header`, která určuje, jestli se jedná o nadpis sekce. Některé překlady tuto možnost obsahují, proto je přítomna i v databázi.

Zdrojový kód 1: index Bible

```
{
  index: 'bible',
  mappings: {
    properties: {
      translation: { type: 'keyword' },

```

```

    book: { type: 'integer' },
    chapter: { type: 'integer' },
    verse: { type: 'integer' },
    text: { type: 'text' },
    is_header: { type: 'boolean' },
  },
},
}

```

5 Implementace

Části projektu jsou rozděleny do samostatných kontejnerů, které spolu komunikují prostřednictvím lokální sítě Dockeru.

5.1 Monorepozitář

Kontejnery tvořící projekt se vytvářejí z kódu a konfigurací uložených v monorepozitářové struktuře. Data jednotlivých kontejnerů jsou rozdělena do balíčků, které jsou reprezentovány podadresáři s příslušnými názvy v adresáři packages.

Hlavní výhodou struktury monorepozitáře je přirozené sdružení více balíčků do jednoho projektu. To umožňuje jednodušší proces sestavení a vydávání nových verzí. Dále je výhodná i možnost sdílení konfigurace a některých závislostí. Díky společnému umístění v projektu lze velmi jednoduše spustit celý projekt v testovacím prostředí a ověřit správnou komunikaci balíčků mezi sebou a případné odstranění chyb. Projekt je také velmi jednoduše přenositelný a na novém zařízení vyžaduje jen minimální konfiguraci.

5.1.1 Nástroje pro správu projektu

Každý balíček má vlastní specifické konfigurace jednotlivých nástrojů. Společné konfigurace, které jsou globální pro projekt nebo sdílené více balíčky, jsou umístěny v hlavním adresáři projektu.

Společná konfigurace zahrnuje především docker compose soubory. Dále zde nalezneme také konfiguraci TypeScriptu pro typovou kontrolu a sestavení projektu a nástrojů pro kontrolu kvality kódu a testování.

5.1.1.1 GIT Nástroj GIT jsem použil pro správu verzí projektu. V CI/CD je verzování klíčové a řídí procesy kontroly kvality a doručení připravených artefaktů uživateli. Více se budu konvencím verzování v projektu věnovat dále v textu.

5.1.1.2 Pnpm Pnpm jsem použil pro správu závislostí projektu a pro vytvoření pnpm skriptů, které usnadňují spouštění složitých příkazů specifických pro

projekt. Tuto možnost jsem využil pro zjednodušení přístupu k nástrojům pro kontrolu kódu, kompilaci a spuštění docker compose.

5.2 Server

Server poskytuje HTTP API pro manipulaci s daty uloženými v Elasticsearch a PostgreSQL databázích. Poskytuje možnost správy uživatelů, sezení, kontrolu oprávnění a implementaci business logiky.

Klíčovým aspektem při návrhu architektury pro mě byla budoucí rozšiřitelnost aplikace. Proto jsem se rozhodl pro využití modulární monolitické architektury, která aplikaci rozděluje do více modulů, které reprezentují jednotlivé části domény aplikace. Moduly jsou od sebe odděleny, což usnadňuje vývoj samotný a umožňuje udržet si přehled o částech, které budou ovlivněny případnými změnami.

Modulární monolit je kombinací výhodných vlastností mikroservisů a tradiční monolitické architektury. Celá aplikace je tak tvořena sdílenými nástroji a express serverem spolu s moduly, do kterých je rozdělena doménová logika aplikace. Modulární monolitická architektura je pro návrh a nastavení složitější, než běžná monolitická architektura, poskytuje však přirozenější rozdělení kódu podle domény, namísto podle technického určení. Zároveň je podstatně jednodušší na správu, než architektura mikroservisů i když poskytuje některé stejné výhody.

Tuto architekturu jsem zvolil i s ohledem na budoucí údržbu a další rozvoj aplikace. V takto strukturovaném kódu se jednodušeji provádějí úpravy, protože části, které jsou spojeny doménou, a tedy zároveň i na sobě závislé, se nacházejí na stejném místě.

5.2.1 Globální nástroje

Jádrem aplikace je sada globálně dostupných nástrojů, které implementují funkcionalitu, které jsou nezávislé na doméně a používají se na více místech napříč aplikací.

Tyto nástroje jsou uloženy ve složce shared.

Patří sem:

- Middleware pro ošetření chyb
- Validátor uživatelských dat v požadavku
- Adaptér pro Elasticsearch
- Adaptér pro PostgreSQL
- Logger

Některým z nich se níže věnuji podrobněji.

5.2.1.1 Ošetření chyb Express.js má vestavěný mechanismus pro ošetření chyb. Pokud dojde k chybě při zpracování HTTP požadavku, zavolá se automaticky výchozí middleware pro ošetření chyb. Pro potřeby tohoto projektu jsem implementoval middleware, který zajišťuje ošetření chyb a jejich převedení na stavové HTTP kódy, které jsou důležité pro uživatele. Vytvořil jsem systém výjimek, které odpovídají stavovým kódům pro odpověď. Všechny tyto výjimky dědí ze třídy `AppError`. K výjimce je možné přidat také textovou zprávu.

Výjimky mohou reprezentovat různé stavy, kdy server požadavek nemůže správně zpracovat. Pro potřeby aplikace jsem využil tyto stavové kódy pro různé nežádoucí stavy. Jejich sémantiku jsem interpretoval podle dokumentace z Mozilla Developer Network[[mdn-status](#)]:

- `AppError(500)`: výchozí výjimka, je možné u ní změnit kód, čehož využívají ostatní výjimky, které z této dědí. Měnit kód ručně je nežádoucí, protože každá z výjimek nese nějakou informaci a tato je nejobecnější. Proto by se měla používat pro neočekávané chyby serveru.
- `ValidationError(400)`: Používá se především v middleware pro validaci uživatelských požadavků. Je vrácena, pokud uživatelův požadavek neodpovídá očekávanému formátu.
- `AuthError(401)`: Oznamuje uživateli, že se pokouší bez platného přihlášení přistoupit k datům, která jsou dostupná jen přihlášeným uživatelům.
- `PermissionError(403)`: Informuje o skutečnosti, že server zná identitu klienta a odmítá mu poskytnout požadovaná data z důvodu nedostatečných oprávnění.
- `NotFoundError(404)`: Tato výjimka je vrácena, pokud nelze najít žádný výsledek, který by odpovídal požadavku.
- `ConflictError(409)`: K této výjimce dojde, pokud uživatel požaduje platnou akci, není však možné ji provést vzhledem k aktuálnímu stavu databáze. Například pokud se pokouší vytvořit záznam v databázi, který by porušoval unikatnost.

Pokud dojde k vyvolání výjimky, přijme ji middleware pro obsluhu chyb, který ověří, jestli jde o instanci třídy `AppError` a pokud ano, vloží data z výjimky do odpovědi. Pokud nejde o instanci `AppError`, vytvoří se odpověď, která ukazuje, že jde o neočekávanou chybu serveru.

5.2.1.2 Middleware pro validaci požadavků Každý HTTP požadavek může obsahovat data od uživatele. Před předáním k dalšímu zpracování je nutné ověřit, že uživatel předal požadavek ve správném formátu. K tomu slouží validator, který ověří formát zvolené části požadavku podle zadaného schématu (vizte ??).

Zdrojový kód 2: Middleware pro validaci požadavků

```
export function requestValidator<T extends ZodType>(
  schema: T,
  part: string = 'params',
) {
  return (req: Request, _res: Response, next:
    NextFunction) => {
    try {
      const parsedData = schema.parse(req[part]);
      (req as ValidatedRequest<z.infer<typeof
        schema>>).validated = parsedData;
      next();
    } catch (error) {
      next(new ValidationError('Bad request format',
        error as Error));
    }
  };
}
```

Validátor je implementován jako funkce vyššího řádu se dvěma parametry. První je zod schéma, které definuje žádaný formát objektu s daty od uživatele. Druhý parametr je textový řetězec, který určuje, jak uživatel data předal. Data mohou být předána buď v těle dotazu, jako součást url adresy, nebo jako HTTP parametry.

Všechny tři možnosti jsou v expressu reprezentovány jako objekt v dotazu. Dotaz validátor přijímá v parametru req. Pokud validace proběhne správně, middleware přetypuje dotaz na generický typ ValidatedRequest<T>, kde T je typ objektu validovaných dat. Typ ValidatedRequest obsahuje vlastnost validated, ve které jsou zvalidovaná data uložena. Díky tomu má již TypeScript typovou kontrolu nad uživatelskými daty a je možné s nimi bezpečně pracovat. Middleware v tom případě volá funkci next, která reprezentuje další funkci, která má požadavek zpracovávat.

Pokud validace selže (vznikne při ní výjimka), Je zavolána funkce next s výjimkou ValidationError, čímž dojde k předání požadavku systému pro ošetření výjimek.

Reálným omezením tohoto návrhu je skutečnost, že není možné předávat uživatelská data v jednom požadavku více metodami zároveň. Vzhledem k aktuálnímu rozsahu projektu jsem však nenarazil na to, že by pro mne byla tato možnost klíčová. V budoucnu je možné tento nedostatek vyřešit přidáním vlastností, které reprezentují jednotlivé metody dovnitř vlastnosti validated.

5.2.1.3 Připojení k databázi

Aplikace komunikuje se dvěma databázemi. PostgreSQL i Elasticsearch mají vlastní správu připojení a rozhraní pro jednodušší vytváření dotazů.

5.2.1.3.1 PostgreSQL Pro připojení k PostgreSQL databázi je použita knihovna Kysely. Knihovna samotná poskytuje jednoduchý přístup ke správě připojení a vytváření dotazů. Ve zbytku projektu je rozhraní knihovny používáno přímo.

5.2.1.3.2 Elasticsearch Pro komunikaci mezi Node.js aplikací a instancí Elasticsearch existuje knihovna stejného názvu. Tato knihovna poskytuje velmi základní rozhraní, proto jsem vytvořil třídu ElasticAdapter, která nad ní vytváří vrstvu abstrakce. Ve třídě ElasticAdapter jsou implementovány metody, které zajišťují vyhledávání, agregaci dat, vkládání dat a další operace. Tyto metody jsou navrženy podle potřeb, které vyvstaly při vývoji projektu.

Komunikace s Elasticsearch probíhá prostřednictvím HTTP požadavků. Knihovna elasticsearch pouze tyto dotazy realizuje a kontroluje jejich strukturu. Třída ElasticAdapter tak zajišťuje ošetření chyb, kontrolu typů a také základní společné nastavení pro jednotlivé skupiny dotazů (např. vyhledávání, hromadné vkládání, vyhledávání s agregací, ...).

5.2.2 Inicializace aplikace

Vstupním bodem pro start aplikace je soubor index.ts v adresáři src tohoto modulu. Odtud se volá funkce initializePostgres. Tato funkce vytvoří v databázi uživatele s administrátorskými oprávněními, pokud v databázi ještě žádný uživatel není. Login a heslo uživatele může uživatel nastavit přes proměnné prostředí. Poté se spustí server, jehož konfigurace je uložena v souboru bootstrap.ts.

5.2.3 Moduly

Aplikace je složena z těchto modulů:

- **auth:** Zajišťuje registraci, přihlášení a odhlášení uživatele. Dále také správu uživatelských rolí a ověření, jestli je uživatel přihlášený, případně jestli má přiřazenou zvolenou roli.
- **bible:** Tento modul poskytuje rozhraní pro přidávání a zobrazování textů Bible a také pro správu úryvků.
- **group:** Zajišťuje správu skupin a jejich členů.
- **psalter:** Zajišťuje práci s texty žaltáře.
- **user:** Slouží ke správě uživatelských dat, což jsou záložky a metadata veršů. Metadata slouží k přidávání poznámek a zvýraznění veršů Bible.

5.2.4 Struktura modulu

Každý modul sdružuje funkcionality, které spadají do stejné domény. Po technické stránce je hlavní struktura modulu jednotná. Hlavní součásti a jejich určení jsou vždy stejné. Hlavní úlohou serveru je poskytovat uživateli přístup k datům v databázi prostřednictvím HTTP rozhraní. Tomu odpovídá i struktura každého modulu.

Stěžejní částí každého modulu je v případě této aplikace takzvaný servis. Servis je třída, která reprezentuje jednotlivé operace v dané části domény. Například pro záložky by šlo o možnost vypsát všechny záložky, přidat novou, nebo upravit existující. Servis tak poskytuje rozhraní, které používá typy reprezentující jednotlivé entity ve zvolené doméně. Toto rozhraní využívá buď controller daného modulu, který zajišťuje mezivrstvu mezi express routerem a business logikou v servisu, nebo servisy ostatních modulů. Tím je zajištěno, že při manipulaci s daty z jiného modulu není potřeba soustředit se na business logiku této sekce. Zároveň jsou tak závislosti mezi jednotlivými moduly značně méně komplikované, což usnadňuje testování i refactoring kódu.

- **Router** mapuje HTTP požadavky na příslušné metody controlleru. Prostřednictvím middlewaru se zde také provádí ověření, jestli je uživatel přihlášený a provádí se validace dat požadavku.
- **Controller** validuje a zpracovává data a požadavky. Zajišťuje komunikaci s routerem a spouští servis. Controller se stará o úlohy specifické pro express. Extrahuje data z Request objektu a nastavuje data, která jsou odeslána zpět uživateli.
- **Service** zajišťuje získání a zpracování dat na základě požadavku. Výsledek vrací controlleru. Zajišťuje komunikaci mezi jednotlivými použitými závislostmi.
- **Repository** zajišťuje komunikaci s databází prostřednictvím adaptéru a poskytuje metody specifické pro danou doménu. Provádí překlad mezi doménovými typy a typy, které reflektují strukturu databáze. Zajišťuje kontrolu a validaci dat. Obstarává konzistenci dat v databázi.

5.3 Frontend

Tento balíček vytváří grafické uživatelské rozhraní nad HTTP API. Slouží k prezentaci dat z API a k jejich snadnější manipulaci.

5.3.1 Progresivní webová aplikace

Technologie progresivních webových aplikací (dále PWA) umožňuje z webové aplikace vytvořit aplikaci spustitelnou na různých platformách. Hlavním cílem pro tuto práci bylo vytvořit aplikaci, která bude pro uživatele snadněji dostupná,

což je usnadněno možností provozovat aplikaci v režimu, který je obdobný nativním mobilním aplikacím.

Konfiguraci PWA jsem realizoval prostřednictvím knihovny `vite-plugin-pwa` [`vite-plugin-pwa`]. Tento plugin se nastavuje v konfiguračním souboru pro Vite.

Minimálními požadavky [`pwa-installation`], aby bylo možné PWA nainstalovat z prohlížeče Google Chrome, je přítomnost následujících atributů v konfiguračním objektu:

- `name`, nebo `short_name`: Používá se pro pojmenování aplikace po nainstalování.
- `icons`: Obsahuje pole cest k ikonám spojeným s aplikací. Pro instalaci jsou zapotřebí alespoň 2 ikony čtvercového tvaru se stranami délky 192px, resp. 512px. Tyto ikony jsou použity pro zástupce na ploše a pro obrazovku, kdy se aplikace načítá po spuštění.
- `display`, nebo `display_override`: Tyto vlastnosti nastavují žádané zobrazení aplikace. Aplikace může být zobrazena v režimu, který vypadá jako prohlížeč, nebo se skrytými ovládacími prvky prohlížeče.
- `start_url`: Označuje výchozí adresu aplikace. Tato stránka je načtena, když uživatel aplikaci otevře.
- `prefer_related_applications`: Pokud existuje nativní aplikace, jejíž instalace je preferována před PWA, touto hodnotou lze toto chování nastavit. Je k tomu vyžadován seznam preferovaných aplikací ve vlastnosti `related_applications`.

5.3.2 Struktura balíčku

Balíček frontend je také rozdělen do modulů. Zde jsem se rozhodl pro méně striktní oddělení jednotlivých modulů, ale pokusil jsem se dodržet rozčlenění podle domény, které přibližně odpovídá rozdělení na straně backendu.

Moduly zde představují logické seskupení funkcionalit, které k sobě patří podle domény, moduly však mohou importovat části z jiných modulů. Pro uchování částí, které jsou velmi často používány z různých míst projektu jsem vytvořil složku `shared`. Zde je umístěn například HTTP klient, vytvořený pomocí knihovny `Axios`. Dále se zde nachází část s konfigurací internacionalizace.

Vzhledem k tomu, že nejdůležitější funkcí tohoto balíčku je zajistit uživatelské rozhraní pro komunikaci s backendem, každý modul obsahuje vlastního HTTP klienta, který používá sdílenou nakonfigurovanou instanci `Axios`. Klienti jednotlivých modulů definují metody, které vytváří jasně definované rozhraní pro realizaci jednotlivých HTTP požadavků (např. přihlášení, načtení knih Bible, vložení záložky,...).

Částí specifickou pro `Vue.js` je takzvaný store. Story jsou do `Vue.js` přidány pomocí knihovny `Pinia`. Story slouží k ukládání a globálnímu sdílení stavu a jeho mutaci (např. uložení informací o uživatelském účtu, oprávnění uživatele,

možnosti odhlášení, nebo přihlášení uživatele). V případě tohoto projektu je informace ve store často propojena s daty v databázi. Proto jsem často využíval přístup, kdy store tvoří rozhraní nad HTTP klientem. Při změně globálního stavu na frontendu tak zároveň posílá požadavek backendu.

Tento postup lze snadno demonstrovat například na modulu zajišťujícím autentizaci. Funkce login, umístěná v souboru authStore.ts provádí pokus o přihlášení uživatele. Přihlášení je silně závislé na validaci ze strany serveru. Metoda přijímá přihlašovací jméno a heslo. Pošle serveru požadavek na přihlášení uživatele. Pokud požadavek proběhne úspěšně a přihlašovací údaje jsou platné, načte funkce ze serveru podrobnosti o uživateli a uloží je do store.

Vue.js používá pro skládání uživatelského rozhraní takzvané komponenty. Komponenta je celek, který sdružuje HTML kód, logiku, pro manipulaci s HTML elementy a CSS styl elementů komponenty. Při implementaci jsem nevyužil možnost vkládat CSS kód. Pro stylování jsem použil knihovnu Tailwind, která používá sadu předpřipravených stylů, které se elementům přiřazují pomocí HTML tříd.

5.4 Initializer

Initializer Zajišťuje iniciální nastavení databází po startu systému. Kontejner si udržuje informace o požadovaném stavu databází. Při startu vykoná migrace PostgreSQL databáze, které nebyly aplikovány. Pro Elasticsearch vytvoří všechny dosud neexistující indexy.

6 DevOps a CI/CD

V této kapitole se budu věnovat popisu prvků z oblasti DevOps a CI/CD, které jsem v této práci použil.

DevOps je filosofie, která je postavena na propojení procesů vývoje, nasazení a provozu software s důrazem na automatizaci.

V tomto projektu jsem se zaměřil především na CI/CD (continuous integration and continuous delivery), což je proces automatizace integrace provedených změn do projektu a automatizované doručení změn zákazníkovi.

Podle myšlenek DevOps jsem zautomatizoval proces sestavení projektu, vytvořil jsem automatizované testy a výpočet pokrytí kódu těmito testy. Do projektu jsem integroval mechanismy pro ověření kvality kódu. Všechny tyto nástroje je možné spustit v projektu pomocí jednoho příkazu. Dále jsem zajistil i vytváření a ukládání logů. Ve zbytku kapitoly se budu věnovat především CI/CD.

6.1 CI/CD pipeline

CI/CD pipeline v mém projektu zajišťuje automatizaci kontroly kódu, sestavení artefaktů nové verze a jejich přidání do kontejnerového repozitáře. Díky tomu je celý proces začleňování změn do projektu rychlejší a spolehlivější.

Základní částí CI/CD pipeline je kontrola kvality kódu. Ta sestává z kontroly formátování kódu, lintování a provedení automatizovaných testů. Pro jednoduchost a přehlednost a také vzhledem k malé velikosti projektu jsem všechny testy nechal spustit při každé kontrole projektu.

Všechny tyto části kontroly může uživatel spustit ručně v lokálním vývojovém prostředí, nebo se spouští automaticky při každém přidání změn do vzdáleného GitHub repozitáře.

Provedené změny lze do repozitáře přidat pomocí pushe (běžné přidání změn, které může provést každý, kdo má oprávnění vkládat změny do dané větve, neprobíhá zde žádný schvalovací proces). Po přidání změn dostane uživatel zpětnou vazbu, jestli kontrola proběhla úspěšně. Dvě hlavní větve (dev a main) jsou však proti tomuto postupu chráněny. Pro přidání změn do těchto větví je potřeba otevřít takzvaný pull-request. Při tom vznikne žádost o začlenění změn, kterou musí správce projektu potvrdit, aby mohla být přidána.

Proces schvalování pull-requestů je momentálně poněkud zdlouhavý a ne zcela dobře využitý vzhledem k tomu, že na projektu pracuji sám. Tak v tuto chvíli pouze sám potvrzuji vlastní změny. Do budoucna však může být výhodný a pro práci ve vícečlenném týmu je velmi důležitý.

Větve dev a main mají v repozitáři zvláštní roli. Zatímco ostatní větve slouží pro běžný vývoj, opravování chyb a přidávání nových možností, větve dev slouží ke kompletaci těchto změn dohromady. Dokončená změna z některé z ostatních větví tak může být přidána pomocí pull-requestu do větve dev.

Proces vydání nové verze je z části ruční, ověření kvality a sestavení však probíhá automatizovaně. Autor nové verze tak musí vzít změny z větve dev a na nové větvi provede změnu číslování verze. Změna se provádí v package.json souborech jednotlivých balíčků projektu, včetně kořenového balíčku. Dále přidá krátkou zprávu shrnující změny v dané verzi do souboru docs/changelog.md.

S takto provedenými změnami se provede pull-request do větve main. Zde je možné jednotlivé změny označit štítkem. Pokud má štítek správný název, provede se sestavení nové verze a jednotlivé obrazy docker kontejnerů jsou přidány do kontejnerového repozitáře, odkud mohou být staženy.

K dispozici jsou dva typy štítků. Jeden typ označuje kandidáta na novou verzi. Při jeho přidání se sestaveným obrazům přidá pouze štítek s označením dané verze. Pokud je přidán běžný štítek verze, přidá se obrazům také štítek latest. Po stažení se tak obrazy automaticky aktualizují na novou verzi všem uživatelům, kteří tento štítek používají.

6.2 Verzování

Verzování je důležitou součástí CI/CD pipeline, protože jednotlivé události se určují právě podle vytvořených objektů ve verzovacím systému. Jde především o commity, větve, pull-requesty a štítky.

Níže budu popisovat jednotlivé konvence, které jsem pro projekt zavedl.

6.2.1 Větev

Větev obsahuje větší množství ucelených změn, které mají společný účel. Může sloužit například pro vývoj celého konkrétního modulu, pro dokumentaci, úpravy infrastruktury, nebo refactoring balíčku, případně pro vývoj konkrétního doménového prvku.

Zvláštní roli mají větve `dev` a `main`, které jsou chráněny před přímým přidáváním commitů. Změny do nich lze přidávat jen pomocí pull-requestů. Větev `dev` slouží ke kompletaci změn z ostatních větví. Změny, které jsou přidány do větve `main` jsou určeny k vydání. Vydání nové verze se provede automaticky po ošitkování commitu.

Každá větev je pojmenována ve formátu `<typ>/<nazev>`, kde `<nazev>` popisuje společný účel změn ve větvi (např. `server-authentication`). `<typ>` přibližuje, jaký technický charakter změny mají.

Povolené typy větví jsou následující:

- **chore:** Slouží pro údržbu projektu, závislostí, infrastruktury. Nepřidává další funkcionalitu, ani neopravuje chyby v kódu
- **feature:** Přidává skupinu funkcionalit do projektu (čtečka Bible, modlitební skupiny, ...)
- **release:** Stará se o poslední změny před vydáním nové verze. Typicky se zde provádí přidání change logu a zvýšení čísla verze.
- **docs:** Provádí se zde přidání a údržba přiložené dokumentace projektu.

6.2.2 Commity

Jednotlivé commity obsahují menší ucelené změny. Jeden commit by neměl obsahovat více změn, které spolu přímo nesouvisí (tematicky, nebo umístěním).

Pojmenování commitu je vždy ve formátu `<typ>: <popis>`, kde `<popis>` stručně vysvětluje hlavní provedené změny v rámci daného commitu. Dále zpráva ke commitu obsahuje i podrobnější popis provedených změn.

Typy commitů jsou určeny pro budoucí implementaci automatického sémantického verzování. Usnadňují také orientaci ve změnách a upřesňují technický význam commitů. Povolené typy jsou tyto:

- **feature:** přidává novou funkcionalitu
- **fix:** opravuje chybu v kódu
- **test:** přidává automatizované testy pro již existující kód
- **docs:** upravuje dokumentaci
- **build:** změny v nastavení projektu, infrastruktury a závislostech

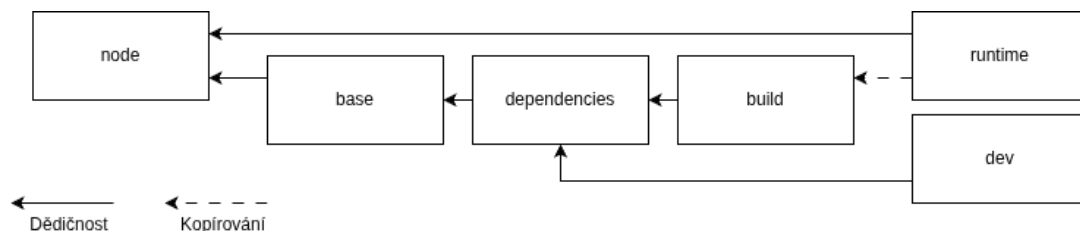
6.2.3 Štítky

GIT poskytuje také štítky, kterými lze označovat jednotlivé body ve stromu verzí. Tuto možnost jsem použil pro automatické vytvoření artefaktů projektu. Štítky se označují jednotlivé verze projektu, které budou vydány. Název štítku obsahuje identifikátor verze (např. v0.1.0) a dále volitelně může název obsahovat část, která určuje, že verze není konečná, ale že jde o kandidáta na verzi (release candidate). Jednotliví kandidáti mají v názvu pořadové číslo. Označení kandidáta na verzi může být např. v0.1.0–rc1. Kandidátů na verzi může být více. Tyto verze slouží k otestování, že se všechny artefakty vytvořily správně a jsou použitelné.

6.3 Sestavení kontejneru

Pro chod aplikace jsou potřeba celkem 3 kontejnery, které jsem vytvořil v rámci tohoto projektu. V této sekci se budu věnovat procesu sestavení obrazu kontejneru pomocí Dockeru. Proces je velmi podobný pro všechny kontejnery. Postup sestavení budu demonstrovat na backendu aplikace.

Celý postup je uložen v soubodu Dockerfile. Proces sestavení obrazu kontejneru je rozdělen do několika stupňů, z nichž každý provádí část procesu. Tyto stupně jsou znovupoužitelné a využívají se podle toho, jestli jde o sestavení produkčního artefaktu, nebo jestli je sestavení prováděno pro účely vývoje a testování.



Obrázek 4: Diagram závislostí stupňů v Dockerfile

Dockerfile je tvořen celkem 5 stupni. Stupně a jejich závislosti je možné vidět na obrázku ???. Stupně jsou rozvrženy tak, aby přehledně oddělovaly jednotlivé části procesu a zároveň efektivněji využívaly cachování. Stupně build i dev dědí ze stupně dependencies, což zajistí, že většina uloženého postupu zůstane uložena z posledního sestavení, pokud zde nedojde ke změnám. Docker umí s těmito změnami pracovat efektivně a kroky, ve kterých nedošlo ke změně si ponechává v cache.

Zde je popis jednotlivých stupňů:

- **base:** Základní stupeň dědí z oficiálního image pro node.js. Provádí konfiguraci a instalaci npm pro správu balíčků.

- **dependencies:** Tento stupeň dědí ze stupně base. Provádí se zde kopírování konfiguračních souborů a instalace závislostí pro vývojové prostředí a kompilaci.
- **build:** V tomto stupni probíhá kopírování zdrojového kódu a jeho kompilace.
- **dev:** Zde se provádí spuštění vývojového serveru. Pro tuto variantu je vyžadováno mít pomocí Dockeru připojený souborový systém obsahující zdrojový kód.
- **runtime:** V tomto stupni probíhá instalace závislostí pro produkční běh a nastavení Nginx serveru.

Specifickým prvkem sestavení obrazu pro frontend aplikace je vložení hodnot z proměnných prostředí. Vite, který zajišťuje sestavení frontendu v kombinaci s Dockerem neposkytuje možnost vkládat proměnné prostředí do kontejneru dynamicky. Proměnné jsou vyhodnoceny při sestavení aplikace. Jako důsledek této skutečnosti jsou proměnné do kontejneru vloženy při kompilaci a sestavení kontejneru a ne z `.env` souboru, jak uživatel předpokládá. Pro kontejnerizaci je toto značný nedostatek, který omezuje šíření již postavených kontejnerů, což je hlavní myšlenkou, kterou má kontejnerizace zajišťovat.

Tento problém jsem vyřešil pomocí shellového skriptu. Tento skript se spouští bezprostředně před startem samotného Nginx serveru, který zajišťuje poskytování souborů uživatelům. Vite při sestavení projektu vkládá konkrétní hodnoty na místa, kde se v kódu odkazují vlastnosti proměnné `import.meta.env`. Ve svém projektu a podle instrukcí poskytnutých v repozitáři[**env-script**], který byl zdrojem použitého skriptu, jsem použil pro sestaví projektu zástupné řetězce. Tyto jsou skriptem následně vyhledány a nahrazeny skutečnými hodnotami, které poskytne uživatel.

6.4 Logování

Uchování a přístup k logům je zajištěn pomocí Docker compose. Pomocí tohoto nástroje je možné záznamy uchovávat, filtrovat je a zobrazovat podle specifikovaného času, nebo konkrétních běžících kontejnerů. Toto je podstatně pohodlnější, než sledovat logy všech běžících služeb na jednom místě v terminálu.

6.5 Spuštění projektu

Výsledné obrazy je možné spustit pomocí souboru `docker-compose.yml`, který je umístěn ve složce `release`. Ten automaticky stáhne obrazy pro nejnovější verzi projektu a spustí je. Pro spuštění na libovolném zařízení s nainstalovaným Dockerem tak stačí spustit ve složce se staženým konfiguračním souborem `docker compose` příkaz `docker compose up`.

Do nastavení projektu je možné zasáhnout pomocí proměnných prostředí. Těm jsem se blíže věnoval v příložené dokumentaci projektu.

7 Ukázka aplikace

V této sekci budou popsány funkce, které aplikace poskytuje. Ukázky jsou zpravidla popsány z pohledu globálního administrátora. Pro ostatní uživatele s nižšími oprávněními je uživatelské rozhraní obdobné, pouze některé funkcionality nejsou dostupné.

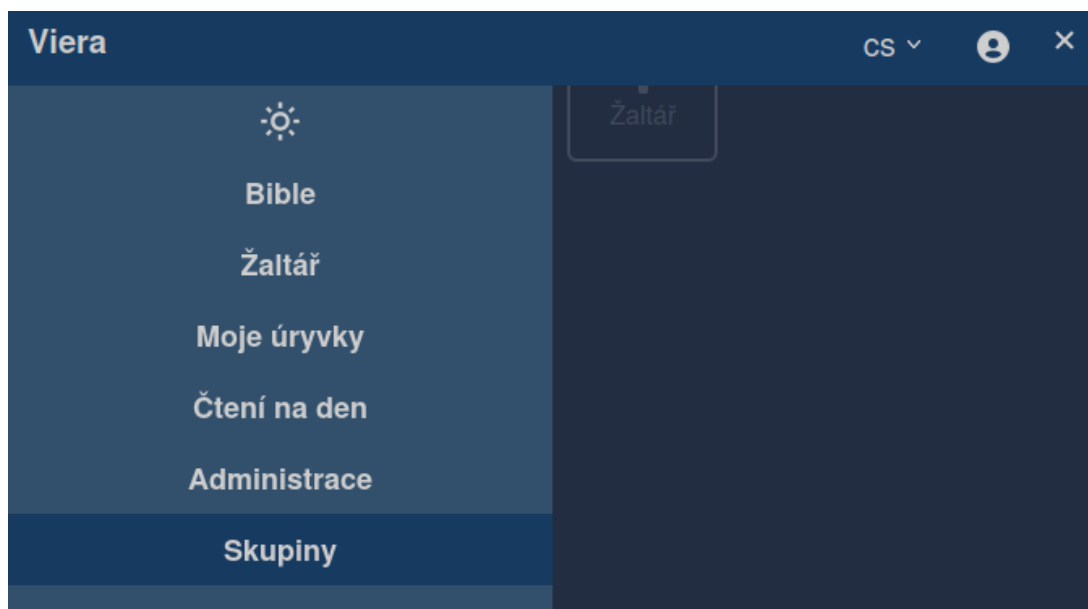
7.1 Navigace

Hlavním ovládacím prvkem je horní lišta (vizte obrázek ??), na které se nachází název aplikace, který funguje také jako odkaz na domovskou stránku. Důležitými ovládacími prvky jsou přepínač jazyka rozhraní, tlačítko, které zobrazuje nabídku pro uživatelský profil, a tlačítko pro otevření bočního menu.



Obrázek 5: Ukázka horní lišty

Pomocí ikonky menu na pravé straně horní lišty lze otevřít boční menu (vizte obrázek ??), které obsahuje přepínač tmavého a světlého motivu rozhraní. Dále jsou zde odkazy na jednotlivé důležité části aplikace. Nabídku lze zavřít pomocí stejného tlačítka, kterým byla otevřena, nebo kliknutím do prostoru okna.

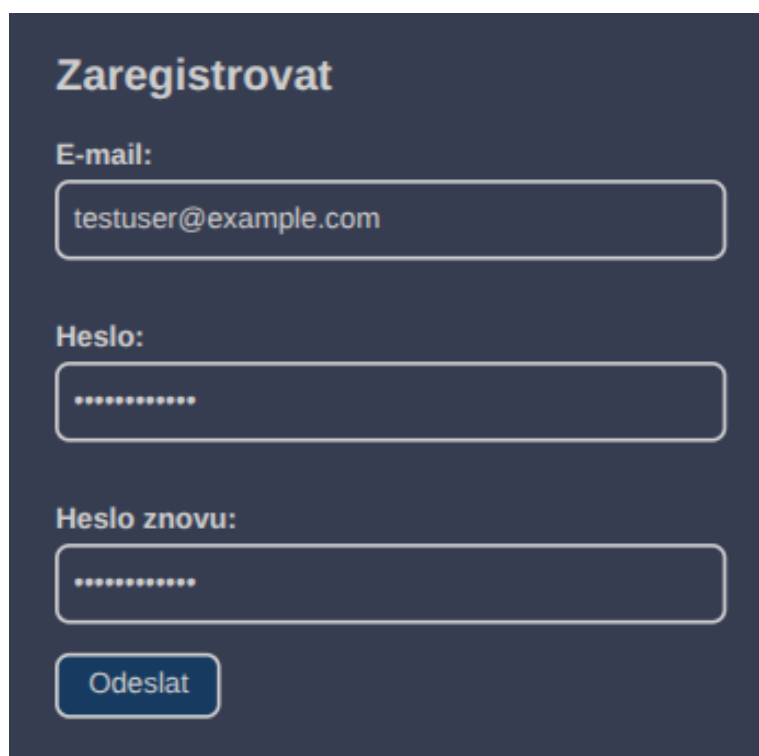


Obrázek 6: Ukázka boční nabídky

7.2 Registrace a přihlášení

Část funkcionalit aplikace je dostupná i nepřihlášeným uživatelům. Nepřihlášení uživatelé mohou procházet globálně dostupné texty (texty Bible, žaltář, úryvky na den). Řada funkcionalit však vyžaduje ukládání uživatelských dat a proto i identifikaci a autentizaci uživatele, aby bylo možné jeho data sdílet mezi zařízeními a ověřovat jeho oprávnění.

Vytvoření uživatelského účtu lze provést prostřednictvím formuláře registrace (vizte obrázek ??). V tomto formuláři stačí vyplnit jen email, který funguje jako unikátní identifikátor uživatelského účtu, a poté vepsat heslo a potvrdit jej opětovným vepsáním do dalšího políčka.



The image shows a registration form on a dark blue background. At the top, the title 'Zaregistrovat' is written in a light blue, sans-serif font. Below the title, there are three input fields and one button. The first field is labeled 'E-mail:' and contains the text 'testuser@example.com'. The second field is labeled 'Heslo:' and contains a series of dots. The third field is labeled 'Heslo znovu:' and also contains a series of dots. At the bottom of the form is a button labeled 'Odeslat'.

Obrázek 7: Ukázka registračního formuláře

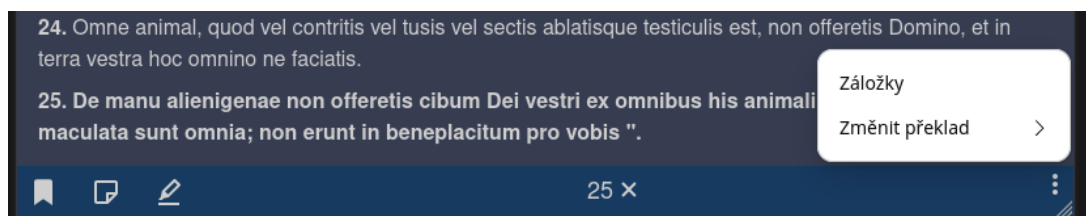
Po registraci je uživatel přesměrován na obrazovku pro přihlášení. Přihlašovací formulář je velmi podobný formuláři pro registraci. Přihlášený uživatel může např. vytvářet vlastní úryvky Bible, vést si poznámky k biblickým textům nebo vytvářet záložky. Může také být členem skupin, ve kterých se zobrazují příspěvky správců skupiny. Pokud jsou mu přidělena administrátorská oprávnění, může mít přístup také ke správě překladů Bible, kalendáře nebo dalších skupin a uživatelských účtů.

7.3 Čtečka Bible

Převážnou část obrazovky s texty Bible vyplňuje samotný zobrazovaný text Bible. Důležitými ovládacími prvky jsou navigační šipky na začátku i na konci textu. Pomocí těch je možné přepínat mezi zobrazenými kapitolami, případně přejít nahoru v Biblické hierarchii (přepnutí na výběr kapitol, nebo knih).

Hlavním ovládacím prvkem čtečky Bible je dolní lišta. V pravém dolním rohu obsahuje tlačítko pro zobrazení nabídky, ve které je možné změnit překlad Bible, nebo přejít do seznamu záložek uživatele.

Do textu mohou přihlášení uživatelé přidávat poznámky k veršům, zvýrazňovat verše a vytvářet pojmenované záložky, ke kterým se mohou později vrátit. Všechny tyto entity jsou spojeny s konkrétním veršem. Pro jejich vytvoření, nebo upravení je nejdříve třeba vybrat z textu konkrétní verš. Poté se aktivují tlačítka v levé části dolní lišty (vizte obrázek ??). Pomocí nich je možné otevřít okna pro správu těchto možností.



Obrázek 8: Ukázka boční nabídky a vybraného verše

7.4 Úryvky Bible

Aplikace obsahuje rozhraní pro tvorbu a upravování úryvků Bible. Každý úryvek je tvořený jednou, nebo více částmi. Začátek a konec každé části se stanovují číslem verše a kapitoly. Kniha Bible je stanovena pro celý úryvek. Úryvky jsou nezávislé na překladu Bible. Překlad se volí podle preferencí uživatele.

7.4.1 Uživatelské úryvky

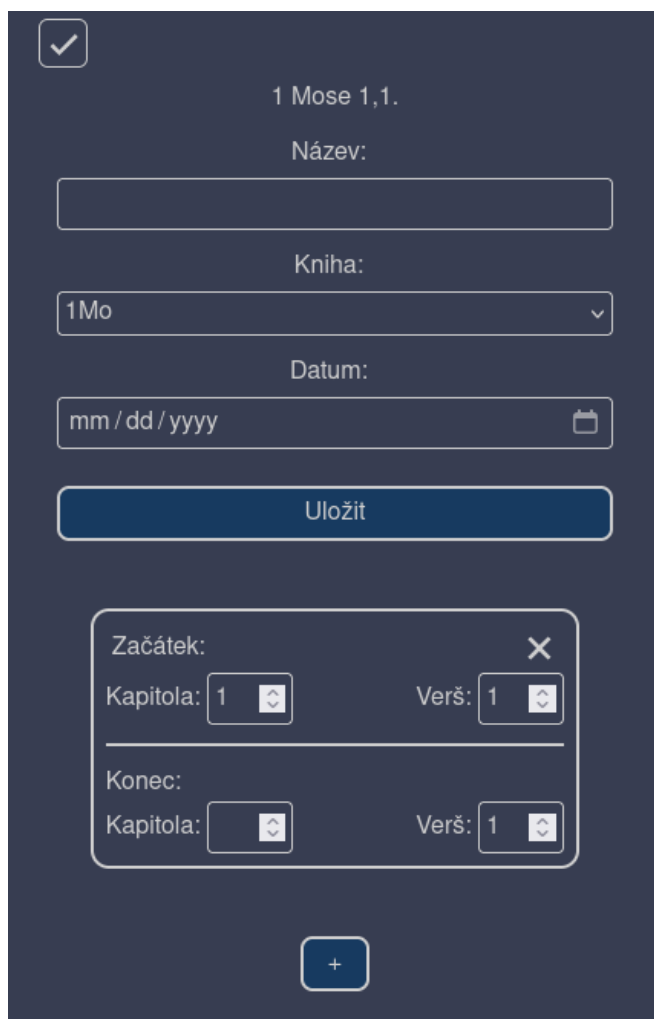
Přihlášený uživatel může vytvářet a spravovat vlastní úryvky. Seznam úryvků se zobrazí po otevření sekce „Moje úryvky“ v boční nabídce. V horní části seznamu úryvků se nachází tlačítko pro vytvoření nového úryvku. Po kliknutí na úryvek se zobrazí pouze vybraný úryvek na samostatné stránce. Odtud je možné dostat se do editoru úryvků stejně jako po kliknutí na tlačítko přidání úryvku.

7.4.2 Editor úryvků

Další důležitou částí je editor úryvků (vizte obrázek ??). Stránka pro upravování úryvků je rozdělena na dvě části. V levé části obrazovky je formulář, do kterého

se vepisují hodnoty, podle kterých se úryvky vytváří. V pravé části je náhled textu, který se aktualizuje podle změn v editoru.

U úryvku je možné nastavit název, knihu a datum. Datum může nastavit pouze uživatel s příslušnými oprávněními. Pokud je datum nastavené, úryvek se bude zobrazovat v kalendáři pro daný den. Pod tlačítkem pro uložení se nachází sekce, ve které lze spravovat jednotlivé části úryvků. Jednotlivé části lze přidávat i odstraňovat. Jejich počet není omezen. U každé části se vyplňuje počáteční a koncový verš a kapitola. Koncová kapitola může zůstat bez hodnoty. V takovém případě aplikace používá stejnou hodnotu jako pro počáteční kapitolu.

The image shows a mobile application interface for editing Bible snippets. At the top, there is a checkmark icon in a square. Below it, the text "1 Mose 1,1." is displayed. The form contains the following elements:

- A label "Název:" followed by a text input field.
- A label "Kniha:" followed by a dropdown menu showing "1Mo".
- A label "Datum:" followed by a date input field with the placeholder "mm / dd / yyyy" and a calendar icon.
- A blue button labeled "Uložit" (Save).
- A section titled "Začátek:" (Start) with a close icon (X) in the top right corner. It contains two rows of dropdowns:
 - Row 1: "Kapitola:" with a dropdown showing "1" and "Verš:" with a dropdown showing "1".
 - Row 2: "Konec:" (End) with a dropdown showing an empty field and "Verš:" with a dropdown showing "1".
- A blue button with a "+" sign at the bottom.

Obrázek 9: Editor úryvků Bible

7.5 Žaltář

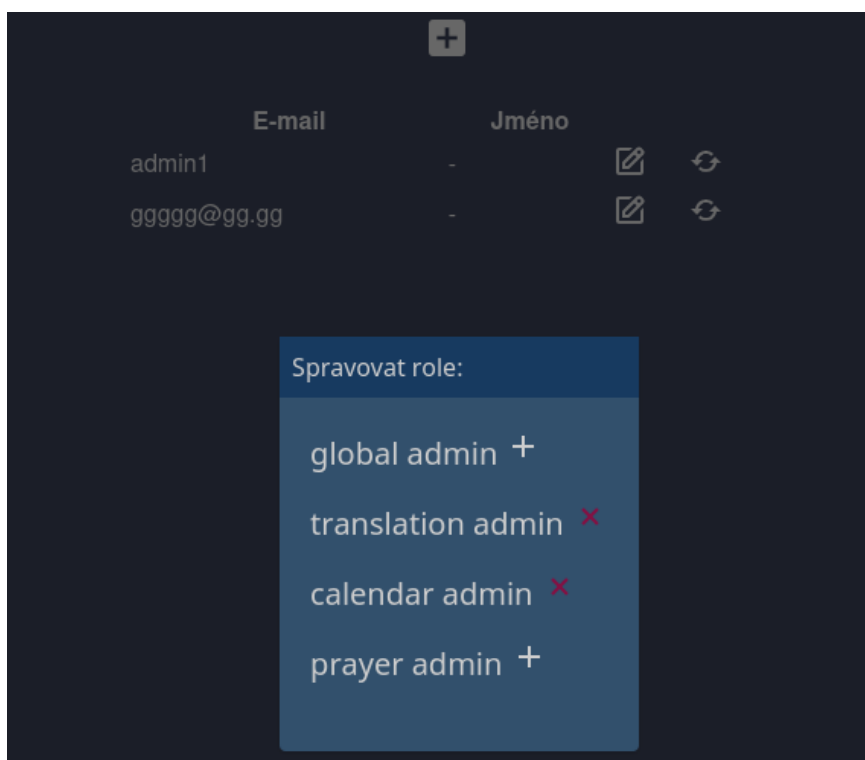
Z bočního menu se lze dostat také do sekce žaltáře. Po kliknutí na odkaz v bočním menu se zobrazí výchozí stránka, na které lze vybrat kaftizmu ze seznamu. V horní části se nachází odkaz na stránku s možností výběru jednotlivých žalmů.

Zobrazení kaftizmy obsahuje jednotlivé žalmy společně s přidruženými modlitbami.

7.6 Administrace

Sekce administrace umožňuje administrátorům spravovat uživatele, jejich role a skupiny, a přidávat překlady textů.

V sekci pro správu uživatelů je možné vytvářet nové uživatele nebo již existujícím uživatelům přidávat a odebírat role. Všichni uživatelé jsou zobrazeni v tabulce. V posledních sloupcích tabulky se nachází tlačítka pro správu rolí uživatele a pro obnovení hesla. Tlačítko pro správu rolí otevře okno se seznamem rolí, které mohou být uživateli přiřazeny (vizte obrázek ??). K jednotlivým rolím v seznamu je přiřazeno tlačítko s akcí, které indikuje, jestli bude kliknutím role přiřazena, nebo odebrána (tím i jestli je momentálně přidělena, nebo ne).



Obrázek 10: Okno pro správu uživatelských rolí

Sekce správy skupin je podobná sekci pro správu uživatelů. Také obsahuje tabulku, ve které jsou vypsány jednotlivé skupiny.

Závěr

V rámci této práce jsem vypracoval webovou aplikaci, která slouží k zobrazení a správě textů Bible a žaltáře. Aplikace také umožňuje vytvářet úryvky z textů Bible, dělat v nich poznámky a záložky. Aplikace obsahuje také sekci pro správu uživatelů a skupin, které umožňují vytváření příspěvků a jejich sdílení s vybranými uživateli.

Hlavním přínosem pro farnost bude možnost přistupovat centralizovaně k těmto textům a dále s nimi pracovat. Pro společenství bude dále také výhodné, že různé překlady textů i uživatelského rozhraní se dají spravovat velmi jednoduše, což v původním řešení, které farnost používala, nebylo uspokojivě zajištěno. V současnosti je žaltář ve farnosti velmi často využíván. Díky této aplikaci bude i spolu s texty Bible dostupný na jednom místě pro všechny farníky i v jejich preferovaném jazyce.

Důraz při vývoji jsem kladl zejména na prvky z oblasti DevOps. Do projektu jsem integroval CI/CD pipeline, automatizované testy, sestavení obrazů kontejnerů a kontrolu kvality kódu, což přispívá k jednoduššímu vývoji a vydávání nových verzí. Také aktualizace aplikace za běhu a nasazení na novém zařízení je díky těmto prvkům značně jednodušší.

V budoucnu se nabízí řada rozšíření aplikace především v oblasti přidání dalších textů modliteb. Textů modliteb řeckokatolické církve je velmi mnoho a i s ohledem na tuto skutečnost jsem se aplikaci snažil navrhnout modulárně. V dalších verzích by mohly být přítomné další bohoslužebné texty i modlitby, které se věřící modlí během dne, nebo kalendář svátků. Pro přidání textů je nejdříve třeba získat jejich potřebné překlady a převést do vhodného formátu, což je značně časově náročné a vyžaduje to spolupráci více lidí. Proto jsem dosud vytvořil jen ukázkovou sadu překladů, která se bude v budoucnu spolu s aplikací rozrůstat.

A Obsah elektronických dat

K textu je přiložena elektronická příloha. Ta obsahuje tato data:

README.md

Obsahem tohoto souboru je popis spuštění projektu a importu připravených ukázkových dat.

text/

Adresář s textem práce ve formátu PDF, vytvořený s použitím závazného stylu KI PřF UP v Olomouci pro závěrečné práce, včetně všech (textových) příloh, a všechny soubory potřebné pro bezproblémové vytvoření PDF dokumentu textu (případně v ZIP archivu), tj. zdrojový text textu a příloh, vložené obrázky, apod.

vera/

Adresář se soubory projektu. Obsahuje zdrojové kódy a dokumentaci a konfigurační soubory projektu, které jsou nutné ke zprovoznění a sestavení aplikace.

vera-data/

Tento adresář obsahuje soubory s ukázkovými daty, které je možné importovat do aplikace. Postup je popsán v textu práce.